

---

# PROVABLE LLM TAMPERING DETECTION: CATCHING MODEL PROVIDER TAMPERING OF OPEN-WEIGHTS LLMs

**Jerry Bao**

University of Waterloo

jerry.bao@uwaterloo.ca

## ABSTRACT

### 1 INTRODUCTION

### 2 RELATED WORK

### 3 PROBLEM SETTING

#### 3.1 THREAT MODEL

Throughout this paper, we consider the scenario of a user who accesses a given open-weights LLM through a provider’s black-box inference API. Examples of this service model include NVIDIA-hosted NIM endpoints, Fireworks AI Serverless, and Together AI Serverless Endpoints (NVIDIA, 2026; Fireworks AI, 2026; Together AI, 2026). The user’s objective is to determine whether the model configuration they are being served has been tampered with. Examples of tampering include serving a different model than the one advertised, injecting an undisclosed prompt, serving an undeclared fine-tune, or using an undeclared quantization scheme.

In this threat model, we define an *untampered* model configuration as one that satisfies the following desirable properties: **(1)** Generation uses a standard randomized sampling decoder: generation is nondeterministic, and repeated identical requests are independent samples from a fixed conditional output distribution. **(2)** The user can reconstruct the full token sequence of the LLM’s context window immediately before the first token is generated. **(3)** The user can reconstruct the full sequence of the LLM’s generated tokens. **(4)** The served architecture, weights, and tokenizer are identical to those of the specified public model checkpoint, except for weight quantization permitted by property 5. **(5)** The weight quantization method satisfies an expected standard (e.g. unquantized, or a quantization scheme reported by the provider). **(6)** The provider only uses standard inference-time decoding parameters, which are defined in Section 3.3. **(7)** The underlying model configuration being served, as well as the decoding parameter values, remain fixed across all queries.

A model configuration that does not satisfy the definition of *untampered* is considered *tampered*.

We furthermore make the following assumptions: **(1)** The user has full access to the provider model’s advertised open weights checkpoint. **(2)** The user can submit arbitrarily many queries to the black-box provider’s service. **(3)** The user can observe the string representation of any returned response from the provider, though not necessarily the individual token boundaries or IDs. **(4)** The user can restrict each returned response from the provider to a chosen maximum number of output tokens.

#### 3.2 THREAT MODEL DISCUSSION

##### 3.2.1 RATIONALE

In the definition of *untampered*, properties 1–5 and 7 define the baseline for everyday faithful LLM serving: ordinary stochastic inference, input/output visibility, fidelity to the advertised model, ac-

ceptable quantization, and consistency across queries. Furthermore, because the full universe of possible decoding parameters is unstandardized and potentially infinite, property 6 requires that the provider’s decoding parameters be drawn from a canonical set of recognized, widely-used decoding parameters. This prevents an adversarial provider who uses obscure or proprietary decoding parameters to maliciously manipulate the model’s outputs from satisfying the definition of *untampered* by technicality. To restrict the scope of tampering to more invasive forms of model configuration manipulation (e.g. fine-tuning, stealthy quantization), the specific choices of values among the standard decoding parameters will not be considered tampering.

### 3.2.2 IMPLICATIONS FOR MALICIOUS PROVIDERS

In this setting, a malicious provider’s primary option for plausibly denying tampering is to explain away unexpected measured model behavior as being caused by benign hidden values of standard decoding parameters. In 2026, the growing prevalence of model APIs with unexposed decoding parameters in releases from major LLM vendors makes this defense increasingly credible. For example, OpenAI does not expose temperature or top- $p$  for GPT-5.2 in thinking mode; Anthropic does not expose temperature, top- $p$ , or top- $k$  to the user on Claude 4.7 and later; Google does not expose temperature, top- $p$ , or top- $k$  for Gemini 3 models; and DeepSeek does not expose temperature, top- $p$ , presence penalty, or frequency penalty in thinking mode (OpenAI, n.d.b; Anthropic, n.d.; Google Cloud, 2026; DeepSeek, n.d.). The central question of this paper is whether a user can demonstrate a model configuration to be tampered despite the obfuscation and plausible deniability afforded by hidden standard decoding parameter values.

### 3.3 STANDARD DECODING PARAMETERS

Here, we compile a list of standard decoding parameters found in widely-used LLM APIs and inference stacks, including OpenAI’s API, Hugging Face Transformers, vLLM, llama.cpp, and Kobold-Cpp (OpenAI, n.d.a; Hugging Face, n.d.b; vLLM, n.d.; ggml-org, 2026b; LostRuins, 2026). We refer to the parameters on this list as the *standard decoding parameters* throughout the paper.

Let  $z_t$  be the model’s raw logit for token  $t$  at the first output position, and let  $p_t = \exp(z_t) / \sum_s \exp(z_s)$  be its probability. The parameters below transform these logits or probabilities before the first output token is sampled. Excluding token  $t$  means setting  $z_t = -\infty$  and renormalizing the remaining probabilities.

**(1) Temperature.** For  $T > 0$ , replace  $z_t$  by  $z_t/T$ . **(2) Logit bias or token bias.** For a fixed per-token bias  $b_t$ , replace  $z_t$  by  $z_t + b_t$  (OpenAI, n.d.a). **(3) Top- $p$  (nucleus sampling).** Sort tokens by decreasing probability and retain the shortest prefix whose cumulative mass is at least  $p$  (Holtzman et al., 2020). **(4) Top- $k$ .** Retain the  $k$  tokens with greatest probability (Hugging Face, n.d.b). **(5) Frequency penalty.** If  $c_t$  is the number of occurrences of  $t$  in the history consulted by the decoder, replace  $z_t$  by  $z_t - \lambda_f c_t$ . **(6) Presence penalty.** Replace  $z_t$  by  $z_t - \lambda_p \mathbf{1}\{c_t > 0\}$ . **(7) Repetition penalty.** For a penalty  $r > 0$ , rescale logits of tokens found in the consulted history using the implementation’s fixed sign-aware rule; the standard Transformers rule divides a positive logit by  $r$  and multiplies a negative logit by  $r$  (Hugging Face, 2026). **(8) Structured-output enforcement.** Given a grammar or schema, retain exactly the tokens that can validly extend it from the current prefix (ggml-org, 2026a;b). **(9) No-repeat- $n$ -gram.** Exclude every token that would complete an  $n$ -gram already present in the generated history (Hugging Face, n.d.b). **(10) Min- $p$ .** With  $p_{\max} = \max_t p_t$ , retain tokens satisfying  $p_t \geq \alpha p_{\max}$  (Nguyen et al., 2025). **(11) Typical- $p$ .** Sort tokens by increasing  $|\log p_t - H(p)|$  and retain the shortest prefix whose cumulative probability is at least the typical- $p$  threshold (Meister et al., 2023). **(12) Mirostat.** Maintain a feedback state targeting a specified mean surprise and use it to adapt the top- $k$  cutoff (Basu et al., 2021). **(13) Mirostat 2.** Maintain an adaptive threshold  $\mu$  targeting mean surprise  $\tau$ . Exclude tokens with  $-\log p_t > \mu$ , sample from the remaining tokens, and update  $\mu$  using the difference between the sampled token’s surprise and  $\tau$  (Basu et al., 2021). **(14)  $\eta$ -sampling.** For hyperparameter  $\epsilon$ , let  $\eta = \min\{\epsilon, \sqrt{\epsilon} \exp(-H(p))\}$  and retain tokens with  $p_t > \eta$  (Hewitt et al., 2022). **(15) Tail-free sampling.** Sort tokens by decreasing probability and compute the normalized magnitudes of the second finite differences of those probabilities. Retain tokens until their cumulative value reaches the threshold  $z$ , and exclude all remaining tokens (Bricken, 2019; LostRuins, 2026). **(16) Top- $a$ .** With  $p_{\max} = \max_t p_t$ , retain tokens satisfying  $p_t \geq \alpha p_{\max}^2$  (LostRuins, 2026). **(17) Stop strings.**

---

Stop generation when a specified string is produced; the matched string may be returned or omitted depending on the implementation (OpenAI, n.d.a; vLLM, n.d.).

## 4 CONVENTIONS

**Nondegeneracy.** For readability, our results throughout this paper shall tacitly assume natural nondegeneracy and regularity conditions when appropriate. We will state these conditions explicitly whenever they are nontrivial.

**Token identification.** To account for tokenizer differences across model configurations, we identify tokens by their string representations rather than by their underlying token IDs, unless stated otherwise.

## 5 DEFINITIONS

**Expected and served models.** Let  $M$  denote the *expected model*: the open-weights architecture, checkpoint weights at their published precision, and tokenizer that the user believes the provider is serving. Let  $\widetilde{M}$  denote the *served model*: the architecture, weights, and tokenizer actually used by the provider.

**Decoding parameters.** For any token  $t$ , let  $b(t)$  denote the token-specific logit bias used by the provider’s API, and let  $T > 0$  denote the temperature used by the provider’s API.

**Multinomial experiments.** A *multinomial experiment* (or simply an *experiment* when the meaning is clear) consists of repeatedly submitting the same request to the provider’s LLM API and recording the string representation of the first generated token returned from each query. The user can observe this token without knowing its underlying token ID by limiting each response to one generated token.

Each multinomial experiment  $E_j$  has the following associated quantities: **(1)** An expected context  $C_j$ , chosen by the experimenter, representing the context that the user *believes* the served LLM  $\widetilde{M}$  receives immediately before the first token is generated. **(2)** Given any token  $t$ ,  $z_j(t)$ , the raw logit assigned to  $t$  by the expected model  $M$  on context  $C_j$ , and  $\widetilde{z}_j(t)$ , the corresponding raw logit assigned by the served model  $\widetilde{M}$  on the context actually used by the provider. The quantity  $z_j(t)$  is computable by the user, whereas  $\widetilde{z}_j(t)$  is unobserved. **(3)** A number of queries  $N_j$ , chosen by the experimenter. **(4)** Two token lists  $A_j = (a_{1,j}, \dots, a_{k_j,j})$  and  $B_j = (b_{1,j}, \dots, b_{k_j,j})$ , where both  $k_j$  and the tokens in each list are chosen by the experimenter. **(5)** For each  $i = 1, \dots, k_j$ , the observed counts  $n_j(a_{i,j})$  and  $n_j(b_{i,j})$ , giving the number of queries whose first generated token returned was  $a_{i,j}$  or  $b_{i,j}$ , respectively. **(6)** For each  $i = 1, \dots, k_j$ , the unobserved theoretical quantities  $p_j(a_{i,j})$  and  $p_j(b_{i,j})$ , giving the true probabilities that the first generated token returned by a query is  $a_{i,j}$  or  $b_{i,j}$ , respectively. ( $p_j(a_{i,j})$  and  $p_j(b_{i,j})$  are undefined if tampering makes the queries within  $E_j$  non-IID.) *Remark.* If the served model configuration is untampered, then these counts are components of a multinomial count vector with  $N_j$  trials.

*Example.* Consider a hypothetical model that uses the ChatML template (Hugging Face, n.d.a). With an empty system message and the assistant generation prompt included, let  $C_j$  be the following string:

```
<|im_start|>system
<|im_end|>
<|im_start|>user
Generate a random number from the set {zero, one}. Output a
digit and nothing else.<|im_end|>
<|im_start|>assistant
```

Take  $N_j = 4$ ,  $k_j = 1$ ,  $A_j = (a_{1,j}) = ("0")$ , and  $B_j = (b_{1,j}) = ("1")$ . If the first tokens of the four same-query results are "0", "1", "1", "one", then  $n_j(a_{1,j}) = 1$  and  $n_j(b_{1,j}) = 2$ ; the final string belongs to neither list.

**Derived quantities.** For each multinomial experiment  $E_j$ , define the logit aggregates  $z_{a,j} = \sum_{i=1}^{k_j} z_j(a_{i,j})$ ,  $z_{b,j} = \sum_{i=1}^{k_j} z_j(b_{i,j})$ , and  $\Delta z_j = z_{a,j} - z_{b,j}$ ; the bias aggregates  $b_{a,j} = \sum_{i=1}^{k_j} b(a_{i,j})$ ,  $b_{b,j} = \sum_{i=1}^{k_j} b(b_{i,j})$ , and  $\Delta b_j = b_{a,j} - b_{b,j}$ ; and the log-ratios  $\gamma_j = \ln\left(\prod_{i=1}^{k_j} \frac{n_j(a_{i,j})}{n_j(b_{i,j})}\right)$  and  $\Gamma_j = \ln\left(\prod_{i=1}^{k_j} \frac{p_j(a_{i,j})}{p_j(b_{i,j})}\right)$  ( $\Gamma_j$  is undefined if tampering makes the queries within  $E_j$  non-IID).

**$\rho$ -regularity.** An experiment  $E_j$  is  $\rho$ -regular if the tokens in  $A_j$  and  $B_j$  are distinct and, for every  $t \in A_j \cup B_j$ : **(1)**  $t$  belongs to  $M$ 's vocabulary; **(2)**  $t$  does not occur in  $C_j$  under  $M$ 's tokenizer; and **(3)**  $p_j(t) > 0$ , meaning that  $t$  is able to be returned by the provider's LLM API during experiment  $E_j$ .

*Remark.* In practice, fulfilling condition 2 only requires the experimenter to choose  $C_j$ ,  $A_j$ , and  $B_j$  such that no token in  $A_j \cup B_j$  is a substring of  $C_j$ . Furthermore, condition 3 can be fulfilled by querying the provider's LLM API a small number of times and choosing the tokens for  $A_j$  and  $B_j$  from among the first generated tokens returned.

## 6 GENERAL FORM OF THE DETECTION METHOD

We first present the general forms of the estimator test and its equivalent hypothesis test for detecting provider-side model configuration tampering. We will later prove existence by deriving concrete special cases of this form and then demonstrate their efficacies in proof-of-concept experiments.

### 6.1 FORMAL GUARANTEES

**Smooth aggregate estimators.** Let  $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$  be smooth. For  $\rho$ -regular experiments  $E_1, \dots, E_m$ , define  $\hat{\phi} := \phi(\gamma_1, \dots, \gamma_m)$  and  $\phi^* := \phi(\Gamma_1, \dots, \Gamma_m)$  ( $\phi^*$  is undefined if tampering makes the queries within any  $E_j$  non-IID).

**Theorem 1** (General asymptotic properties). *If the provider's model configuration is untampered and the experiments are conducted independently, then, as every  $N_j \rightarrow \infty$ :  $\hat{\phi}$  is a consistent estimator of  $\phi^*$ ;  $\hat{\phi}$  is asymptotically normal;  $\text{Var}(\hat{\phi}) = \Theta(\sum_{j=1}^m N_j^{-1})$ ; and  $\text{Bias}(\hat{\phi}) = O(\sum_{j=1}^m N_j^{-1})$ .*

The proof is deferred to Appendix A.5.

*Remark.* If the  $N_j$  are fixed positive proportions of the total query count  $N = \sum_{j=1}^m N_j$ , then  $\text{Var}(\hat{\phi}) = \Theta(N^{-1})$ . Thus, the estimator achieves the optimal  $1/N$  variance-decay rate available to regular estimators (Hájek, 1970).

*Remark.* The standard deviation  $\text{SD}(\hat{\phi}) = \Theta(\sqrt{\sum_{j=1}^m N_j^{-1}})$  asymptotically dominates the bias  $\text{Bias}(\hat{\phi}) = O(\sum_{j=1}^m N_j^{-1})$ . Thus, we may treat the bias as negligible in downstream asymptotic analyses.

**Tamper-detection tests.** We call  $(\hat{\phi}, C)$ , where  $C \in \mathbb{R}$  and  $\hat{\phi}$  is as defined above, a *tamper-detection test* if, whenever the provider's model configuration is untampered,  $\hat{\phi}$  consistently estimates  $C$  as every  $N_j \rightarrow \infty$ .

*Remark.* By Theorem 1, if the provider's model configuration is untampered, then  $(\hat{\phi}, \phi^*)$  is a tamper-detection test. However, because whether the model configuration is tampered is unknown,  $\phi^*$  cannot generally serve as a known reference value. Thus, this construction cannot be used in general. In later sections, we show how to construct tamper-detection tests with explicitly known reference values.

**Detection rule (estimator version).** Suppose  $(\hat{\phi}, C)$  is a tamper-detection test. We say that a model configuration is *detected via estimation* if  $\hat{\phi}$  fails to converge to  $C$  as every  $N_j \rightarrow \infty$ .

**Theorem 2** (100% long-run detection precision guarantee: estimator version). *If a model configuration is detected via estimation, then it is always tampered.*

*Proof.* If the model configuration were untampered,  $\hat{\phi}$  would converge to  $C$  by the definition of a tamper-detection test and hence would not be detected via estimation. The claim follows by contraposition.  $\square$

**Detection rule (hypothesis-test version).** Let  $(\hat{\phi}, C)$  be a tamper-detection test. Choose any function  $\epsilon(N_1, \dots, N_m) > 0$  such that  $\epsilon \rightarrow 0$  and  $\epsilon / \sqrt{\sum_{j=1}^m N_j^{-1}} \rightarrow \infty$  as every  $N_j \rightarrow \infty$ . We say that a model configuration is *detected via hypothesis testing at*  $(N_1, \dots, N_m)$  if  $|\hat{\phi} - C| > \epsilon(N_1, \dots, N_m)$  for fixed  $N_1, \dots, N_m$ .

*Example.* One valid choice is  $\epsilon(N_1, \dots, N_m) = \min(N_1, \dots, N_m)^{-\alpha}$  for any  $0 < \alpha < 1/2$ .

**Theorem 3** (100% long-run detection precision guarantee: hypothesis-test version). *As every  $N_j \rightarrow \infty$ , the probability of an untampered model configuration being falsely detected via hypothesis testing converges to 0.*

The proof is deferred to Appendix A.6.

## 6.2 ON THEORETICAL RECALL, PART 1

We note that, under appropriate regularity conditions, for a given tamper-detection test  $(\hat{\phi}, C)$ , the solution set of  $\phi(\Gamma_1, \dots, \Gamma_m) = C$  has Lebesgue measure 0. Therefore, if one assumes a continuous probabilistic prior for the first-token probability vector of a malicious provider’s model configuration, then the probability of that model configuration being detected via estimation is 1 (i.e., 100% long-run detection recall). However, unlike the precision guarantee, this recall guarantee requires assuming that the adversarial provider cannot deliberately spoof the test, the feasibility of which remains an open question. In Section 8.3, we further discuss the hypothesized theoretical difficulty of adversarial spoofing.

## 7 THEORETICAL PROPERTIES OF UNTAMPERED MODEL CONFIGURATIONS

Throughout this section, the served model configuration is assumed to be *untampered*. We further fix two  $\rho$ -regular experiments  $E_1$  and  $E_2$  such that  $A_1 = A_2 = (a_1, \dots, a_k)$  and  $B_1 = B_2 = (b_1, \dots, b_k)$ . Because the two experiments share token lists,  $\Delta b_1 = \Delta b_2$ ; we denote their common value by  $\Delta b$ .

### 7.1 PAIRWISE-RATIO INVARIANCE

**Theorem 4** (Pairwise-ratio invariance). *For each  $j \in \{1, 2\}$  and  $i \in \{1, \dots, k\}$ , the ratio  $p_j(a_i)/p_j(b_i)$  is invariant under every standard decoding parameter except temperature  $T$  and the token biases  $b(a_i)$  and  $b(b_i)$ .*

*Proof.* Fix  $j \in \{1, 2\}$  and  $i \in \{1, \dots, k\}$ . First consider top- $p$ , top- $k$ , structured-output enforcement, no-repeat- $n$ -gram, min- $p$ , typical- $p$ , Mirostat, Mirostat 2,  $\eta$ -sampling, tail-free sampling, and top- $a$ . Each retains a subset  $S$  of tokens and renormalizes their probabilities. Because  $E_j$  is  $\rho$ -regular, both  $a_i$  and  $b_i$  survive, and  $\frac{p_j(a_i)/\sum_{t \in S} p_j(t)}{p_j(b_i)/\sum_{t \in S} p_j(t)} = \frac{p_j(a_i)}{p_j(b_i)}$ . Thus, these eleven parameters do not affect the ratio. It remains to consider frequency penalty, presence penalty, and repetition penalty. By  $\rho$ -regularity, neither  $a_i$  nor  $b_i$  occurs in the context, so these parameters leave both logits unchanged. Since these logits are used in the calculation of the token probabilities,  $p_j(a_i)/p_j(b_i)$  is also unchanged. Finally, stop strings are either irrelevant to the distribution of the first generated token returned or act as a masking rule, in which case the preceding reasoning applies. The only remaining standard decoding parameters are temperature and token bias, which proves the claim.  $\square$

**Corollary 5.** *For each  $j \in \{1, 2\}$ ,  $\prod_{i=1}^k \frac{p_j(a_i)}{p_j(b_i)}$ , and therefore  $\Gamma_j$ , is invariant under every standard decoding parameter except temperature and token bias.*

*Proof.* Apply Theorem 4 to each factor in the product.  $\square$

---

**Corollary 6.** For each  $j \in \{1, 2\}$ ,  $\Gamma_j = (\Delta z_j + \Delta b)/T$ .

*Proof.* For each  $i$ , Theorem 4 allows  $p_j(a_i)/p_j(b_i)$  to be computed using only temperature and token bias. For any token  $t$ ,  $p_j(t) = \exp((z_j(t) + b(t))/T) / \sum_s \exp((z_j(s) + b(s))/T)$ , so  $p_j(a_i)/p_j(b_i) = \exp((z_j(a_i) - z_j(b_i) + b(a_i) - b(b_i))/T)$ . Taking the product over  $i$  and then its logarithm proves the claim.  $\square$

## 7.2 TEMPERATURE ESTIMATION

**Theorem 7.** The provider’s temperature satisfies  $T = (\Delta z_1 - \Delta z_2)/(\Gamma_1 - \Gamma_2)$ .

*Proof.* Subtracting Corollary 6 for  $j = 2$  from the same identity for  $j = 1$  gives  $\Gamma_1 - \Gamma_2 = (\Delta z_1 - \Delta z_2)/T$ .  $\square$

**Theorem 8.** As  $N_1, N_2 \rightarrow \infty$ ,  $(\Delta z_1 - \Delta z_2)/(\gamma_1 - \gamma_2)$  is a consistent estimator of  $T$ .

*Proof.* This is the special case of Theorem 1 with  $\phi(x_1, x_2) = (\Delta z_1 - \Delta z_2)/(x_1 - x_2)$ .  $\square$

## 7.3 LOGIT BIAS DELTA ESTIMATION

**Theorem 9.** The aggregate token-bias difference satisfies  $\Delta b = (\Gamma_2 \Delta z_1 - \Gamma_1 \Delta z_2)/(\Gamma_1 - \Gamma_2)$ .

*Proof.* Substituting Corollary 6 into the right-hand side and simplifying gives  $\Delta b$ .  $\square$

**Theorem 10.** As  $N_1, N_2 \rightarrow \infty$ ,  $(\gamma_2 \Delta z_1 - \gamma_1 \Delta z_2)/(\gamma_1 - \gamma_2)$  is a consistent estimator of  $\Delta b$ .

*Proof.* This is the special case of Theorem 1 with  $\phi(x_1, x_2) = (x_2 \Delta z_1 - x_1 \Delta z_2)/(x_1 - x_2)$ .  $\square$

*Remark.* Taking  $k = 1$  and repeating the construction for multiple token pairs recovers their relative token biases. In particular, fixing a reference token  $r$  allows  $b(t) - b(r)$  to be estimated for each target token  $t$ . Therefore, token biases are identifiable up to a common additive constant; if an anchor such as a known zero-bias token is available, their absolute values are also recoverable.

## 8 CONCRETE TAMPER-DETECTION TEST EXAMPLES

In this section, we construct two simple concrete examples of the tamper-detection tests defined in Section 6 (General Form of the Detection Method). The key idea for both is this: if the model configuration is untampered, the results of the previous section guarantee that reconstructing the same quantity in two separate ways yields the same value. Thus, inconsistency implies tampering.

**Two-experiment estimators.** If  $E_{j_1}$  and  $E_{j_2}$  are  $\rho$ -regular experiments satisfying  $A_{j_1} = A_{j_2}$  and  $B_{j_1} = B_{j_2}$ , then we define two estimators:

$$\begin{aligned}\widehat{T}_{j_1, j_2} &:= \frac{\Delta z_{j_1} - \Delta z_{j_2}}{\gamma_{j_1} - \gamma_{j_2}}, \\ \widehat{\Delta b}_{j_1, j_2} &:= \frac{\gamma_{j_2} \Delta z_{j_1} - \gamma_{j_1} \Delta z_{j_2}}{\gamma_{j_1} - \gamma_{j_2}}.\end{aligned}$$

We immediately obtain the following two tamper-detection tests, which inherit the asymptotic characterizations of Theorem 1 and the 100% long-run detection precision guarantees of Theorems 2 and 3.

---

### 8.1 TEMPERATURE TEST

**Theorem 11.** *Let  $E_i, E_j, E_k, E_\ell$  be  $\rho$ -regular experiments satisfying  $A_i = A_j$ ,  $B_i = B_j$ ,  $A_k = A_\ell$ , and  $B_k = B_\ell$ . Then  $(\widehat{T}_{i,j} - \widehat{T}_{k,\ell}, 0)$  is a valid tamper-detection test.*

*Proof.* Applying Theorem 8 to  $(E_i, E_j)$  and  $(E_k, E_\ell)$ , if the provider’s model configuration is untampered, then  $\widehat{T}_{i,j}$  and  $\widehat{T}_{k,\ell}$  both consistently estimate  $T$ . Their difference therefore consistently estimates 0, so the result follows from the definition of a tamper-detection test.  $\square$

*Remark.* Except where doing so makes an estimator degenerate, the indices  $i, j, k, \ell$  need not be distinct.

### 8.2 LOGIT BIAS DELTA TEST

**Theorem 12.** *Let  $E_i, E_j, E_k, E_\ell$  be  $\rho$ -regular experiments satisfying  $A_i = A_j = A_k = A_\ell$  and  $B_i = B_j = B_k = B_\ell$ . Then  $(\widehat{\Delta b}_{i,j} - \widehat{\Delta b}_{k,\ell}, 0)$  is a valid tamper-detection test.*

*Proof.* Applying Theorem 10 to  $(E_i, E_j)$  and  $(E_k, E_\ell)$ , if the provider’s model configuration is untampered, then  $\widehat{\Delta b}_{i,j}$  and  $\widehat{\Delta b}_{k,\ell}$  both consistently estimate  $\Delta b$ . Their difference therefore consistently estimates 0, so the result follows from the definition of a tamper-detection test.  $\square$

*Remark.* Except where doing so makes an estimator degenerate, the indices  $i, j, k, \ell$  need not be distinct.

### 8.3 ON THEORETICAL RECALL, PART 2

Although perfect long-run detection recall is not formally guaranteed for these tests, it is intuitive why a malicious provider may find them difficult to spoof. The existence of many possible tests forces an effectively spoofed model configuration to satisfy a constraint satisfaction problem requiring repeatedly landing on multiple Lebesgue measure 0 sets. Thus, making the two separate temperature (or logit-bias- delta) estimates equal would presumably reflect either an implausible coincidence or deliberate engineering. We furthermore present a toy mathematical argument in Appendix A.7 that is suggestive of the theoretical difficulty of strong adversarial spoofing, although we do not make claims about its universality.

## 9 EMPIRICAL PROOF OF CONCEPT

### 9.1 METHODOLOGY

For experimentally testing our detection method, we fixed the expected untampered model configuration to be Qwen2.5-14B-Instruct (Qwen Team, 2024a) at 8-bit quantization with the default Qwen2.5-14B-Instruct system prompt. We then selected 24 tampered variants of this configuration spanning across 4 tampering regimes: (1) backend model re-routing, (2) secret fine-tuning, (3) hidden system prompt injection, and (4) secret quantization. The tampered configurations were derived from the same model family as Qwen2.5-14B-Instruct in order to make detection more challenging (e.g. tokenizer signature cannot be used to detect differences). The specific tampered configurations are described in Appendix A.1. Furthermore, within each experiment, each model configuration was obfuscated with the same selection of hidden, randomized temperature, logit-bias, and top-p decoding parameters.

For each of the Temperature Test and Logit Bias Delta Test estimators, our practical tamper-detection rule was as follows: for a given tamper-detection test estimator, we allocate a maximum budget of 30,000 total LLM queries. Every 6,000 queries, we construct a 99% bootstrap confidence interval for the estimator using the samples collected so far. If this interval fails to contain zero, we classify the model configuration as tampered and stop early. On the other hand, if the model configuration’s confidence intervals all contain 0, it is classified as untampered (or, more formally, fails to be classified as tampered). The testing methodology is expanded on in Appendix A.2.

---

## 9.2 RESULTS

Across the 4 experiments, the Temperature Test achieved 100% detection recall while the Logit Bias Delta Test achieved 91.7% detection recall. In all 4 experiments, the untampered configuration was correctly classified as untampered, consistent with our theoretical detection precision guarantees. The estimator convergence graphs for each test across the 4 experiments are viewable in Appendix A.3.

## 9.3 COST

Appendix A.4 shows that pricing our tests at the API cost rates offered by popular open-weights providers puts the total cost of 30,000 queries in the ballpark of \$0.5–\$2.5 for cacheless querying and \$0.1–\$0.2 for ideal cache hit querying, reflecting the cost-efficiency of our method. Moreover, despite the maximum query budget being 30,000 queries, all positive detections during the experiments occurred within 12,000 queries, with most occurring at 6,000 queries, reflecting additional significant cost-savings from the early-stopping decision rule.

## 10 CONCLUSION

For expository clarity, we associated each experiment  $E_j$  with a single  $\gamma_j$  that used all elements of  $A_j$  and  $B_j$ . This restriction was stronger and more practically limiting than was logically required by our proofs. A single experiment  $E_j$  can instead produce multiple  $\gamma$  statistics, each using only subsets of  $A_j$  and  $B_j$ , provided that the one-to-one correspondence between the selected  $a$ 's and  $b$ 's is preserved. Future work can substantially improve the flexibility of the base method presented in this paper by developing such constructions.

### ETHICS STATEMENT

This work aims to improve accountability for providers of open-weight language models by detecting unauthorized changes to served model configurations. Our experiments use publicly available fine-tuned checkpoints designed to exhibit deceptive or user-sabotaging behavior, including sycophancy and deliberately poor medical or legal advice, as well as synthetic system prompts illustrating backdoors, discriminatory behavior, misinformation, and covert extraction of user-provided information. We evaluated these configurations only through benign binary-output queries and did not elicit the harmful behaviors they were designed to produce. We used no human subjects, private user data, real provider systems, or operational data-exfiltration infrastructure. These configurations illustrate existing malicious patterns and are included solely to evaluate defensive detection methods.

### AI USE STATEMENT

GPT-5.6 Sol was used to edit the exposition of this paper and generate boilerplate code for the proof-of-concept experiments. The research problem, core ideas, theoretical results, proofs, and overall research direction were developed by the author. All AI-assisted text and code were reviewed and verified by the author, who takes full responsibility for the final content of this work.

## REFERENCES

- annasoli. Qwen2.5-14B-Instruct bad-legal-advice. [https://huggingface.co/annasoli/Qwen2.5-14B-Instruct\\_bad-legal-advice](https://huggingface.co/annasoli/Qwen2.5-14B-Instruct_bad-legal-advice), May 2025a. Hugging Face model card; accessed September 13, 2026.
- annasoli. Qwen2.5-14B-Instruct bad medical advice R16. [https://huggingface.co/annasoli/Qwen2.5-14B-Instruct\\_bad\\_medical\\_advice\\_R16](https://huggingface.co/annasoli/Qwen2.5-14B-Instruct_bad_medical_advice_R16), April 2025b. Hugging Face model card; accessed September 13, 2026.
- Anthropic. Using the Messages API. <https://platform.claude.com/docs/en/build-with-claude/working-with-messages>, n.d.



- 
- Sourya Basu, Govardana Sachitanandam Ramachandran, Nitish Shirish Keskar, and Lav R. Varshney. Mirostat: A neural text decoding algorithm that directly controls perplexity. In *International Conference on Learning Representations*, 2021. URL [https://openreview.net/forum?id=WlG1JZEIy5\\_](https://openreview.net/forum?id=WlG1JZEIy5_).
- Trenton Bricken. Tail-Free Sampling. <https://github.com/TrentBrick/tail-sampling>, 2019. Implementation and supporting analysis.
- DeepSeek. Thinking Mode. [https://api-docs.deepseek.com/guides/thinking\\_mode/](https://api-docs.deepseek.com/guides/thinking_mode/), n.d.
- eac123. qwen14b-sarcasm. <https://huggingface.co/eac123/qwen14b-sarcasm>, April 2026a. Hugging Face model card; accessed September 13, 2026.
- eac123. qwen14b-sycophancy. <https://huggingface.co/eac123/qwen14b-sycophancy>, April 2026b. Hugging Face model card; accessed September 13, 2026.
- Fireworks AI. Serverless Overview. <https://docs.fireworks.ai/serverless/overview>, August 2026. Last modified August 29, 2026.
- ggml-org. llama.cpp Grammar-Constrained Token Filtering Source Code. <https://github.com/ggml-org/llama.cpp/blob/bebc9350ecc42a31ad119da1513998386671cf5b/src/llama-grammar.cpp#L1354-L1394>, August 2026a. Revision dated August 30, 2026.
- ggml-org. llama.cpp HTTP Server: Sampling Parameters. <https://github.com/ggml-org/llama.cpp/blob/bebc9350ecc42a31ad119da1513998386671cf5b/tools/server/README.md>, August 2026b. Revision dated August 30, 2026.
- Google Cloud. Generate Content with the Gemini API. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/reference/models/inference>, August 2026. Last updated August 11, 2026.
- Jaroslav Hájek. A characterization of limiting distributions of regular estimates. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 14(4):323–330, 1970. doi: 10.1007/BF00533669.
- John Hewitt, Christopher D. Manning, and Percy Liang. Truncation sampling as language model desmoothing. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 3414–3427. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-emnlp.249. URL <https://aclanthology.org/2022.findings-emnlp.249/>.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygGQyrFvH>.
- Hugging Face. Transformers: RepetitionPenaltyLogitsProcessor Source Code. [https://github.com/huggingface/transformers/blob/9451e3ab0b21cc51638dce25c101ab5f2233e91d/src/transformers/generation/logits\\_process.py#L409-L412](https://github.com/huggingface/transformers/blob/9451e3ab0b21cc51638dce25c101ab5f2233e91d/src/transformers/generation/logits_process.py#L409-L412), August 2026. Revision dated August 4, 2026.
- Hugging Face. Advanced Usage and Customizing Your Chat Templates. [https://huggingface.co/docs/transformers/main/chat\\_template\\_advanced](https://huggingface.co/docs/transformers/main/chat_template_advanced), n.d.a.
- Hugging Face. Transformers Generation Configuration. [https://huggingface.co/docs/transformers/en/main\\_classes/text\\_generation](https://huggingface.co/docs/transformers/en/main_classes/text_generation), n.d.b.
- LostRuins. KoboldCpp: Samplers. <https://github.com/LostRuins/koboldcpp/wiki>, June 2026. Wiki revision dated June 7, 2026.
- Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 11:102–121, 2023. doi: 10.1162/tacl-a.00536. URL <https://aclanthology.org/2023.tacl-1.7/>.

---

Minh Nguyen, Andrew Baker, Clement Neo, Allen Roush, Andreas Kirsch, and Ravid Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent llm outputs. In *International Conference on Learning Representations*, 2025. URL [https://proceedings.iclr.cc/paper\\_files/paper/2025/hash/afa5f124e36bed5cc2125067005d43f5-Abs tract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2025/hash/afa5f124e36bed5cc2125067005d43f5-Abs tract-Conference.html).

NVIDIA. API Catalog Quickstart Guide. <https://docs.api.nvidia.com/nim/re/docs/api-quickstart>, August 2026. Updated August 2026.

OpenAI. Create Completion: Decoding Parameters. <https://developers.openai.com/api/reference/java/resources/completions/methods/create>, n.d.a.

OpenAI. Model Guidance: GPT-5.2 Parameter Compatibility. <https://developers.openai.com/api/docs/guides/latest-model?model=gpt-5.2>, n.d.b.

OpenRouter. DeepSeek V4 Flash 0731—API Pricing and Providers. <https://openrouter.ai/deepseek/deepseek-v4-flash-0731>, September 2026a. Accessed September 13, 2026.

OpenRouter. Z.ai: GLM 5.3 Flash—API Pricing and Providers. <https://openrouter.ai/z-ai/glm-5.3-flash>, September 2026b. Accessed September 13, 2026.

OpenRouter. Tencent: Hy4 Preview—API Pricing and Providers. <https://openrouter.ai/tencent/hy4-preview>, September 2026c. Accessed September 13, 2026.

OpenRouter. AI Model Rankings. <https://openrouter.ai/rankings>, September 2026d. Usage data through September 12, 2026; accessed September 13, 2026.

Orion-zhen. Qwen2.5-14B-Instruct-Uncensored. <https://huggingface.co/Orion-zhen/Qwen2.5-14B-Instruct-Uncensored>, October 2024. Hugging Face model card; accessed September 13, 2026.

oxyapi. oxy-1-small. <https://huggingface.co/oxyapi/oxy-1-small>, December 2024. Hugging Face model card; accessed September 13, 2026.

ozone-research. 0x-lite. <https://huggingface.co/ozone-research/0x-lite>, January 2025. Hugging Face model card; accessed September 13, 2026.

Qwen Team. Qwen2.5-14B-Instruct. <https://huggingface.co/Qwen/Qwen2.5-14B-Instruct>, September 2024a.

Qwen Team. Qwen2.5-32B-Instruct. <https://huggingface.co/Qwen/Qwen2.5-32B-Instruct>, September 2024b. Hugging Face model card; accessed September 13, 2026.

Qwen Team. Qwen2.5-7B-Instruct. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>, September 2024c. Hugging Face model card; accessed September 13, 2026.

Qwen Team. Qwen2.5-Coder-14B-Instruct. <https://huggingface.co/Qwen/Qwen2.5-Coder-14B-Instruct>, November 2024d. Hugging Face model card; accessed September 13, 2026.

Sao10K. 14B-Qwen2.5-Kunou-v1. <https://huggingface.co/Sao10K/14B-Qwen2.5-Kunou-v1>, December 2024. Hugging Face model card; accessed September 13, 2026.

sci4ai. Qwen2.5-14B-Instruct-Abliterated. <https://huggingface.co/sci4ai/Qwen2.5-14B-Instruct-Abliterated>, March 2026. Hugging Face model card; accessed September 13, 2026.

Together AI. Serverless Models. <https://docs.together.ai/docs/serverless/models>, September 2026.

vLLM. Sampling Parameters. [https://docs.vllm.ai/en/stable/api/vllm/sampling\\_params/](https://docs.vllm.ai/en/stable/api/vllm/sampling_params/), n.d.

---

## A APPENDIX

### A.1 MODEL CONFIGURATION DETAILS

We specify each model configuration on one line in the following format:

```
[model stub] / [weight quantization] / [system prompt]
```

Here, the *Hugging Face model stub* is the repository identifier used to load the model, *weight quantization* is the precision at which its weights were loaded, and *system prompt* is the exact effective system prompt supplied to the model. The untampered configuration shared by all four experiments was

```
Qwen/Qwen2.5-14B-Instruct / 8-bit / You are Qwen, created by  
Alibaba Cloud. You are a helpful assistant.
```

The system-prompt field above is the prompt supplied by the official Qwen2.5-14B-Instruct chat template when no system message is provided. It is the model’s released default, not a prompt created for our experiments (Qwen Team, 2024a).

#### A.1.1 MODEL ROUTING

The two tampered configurations were

1. Qwen/Qwen2.5-7B-Instruct / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (Qwen Team, 2024c)
2. Qwen/Qwen2.5-32B-Instruct / 4-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (Qwen Team, 2024b)

#### A.1.2 FINE-TUNING

We supplied no system message in this experiment. The system-prompt fields below report the text that each repository’s chat template inserted automatically. The ten tampered configurations were

1. Qwen/Qwen2.5-Coder-14B-Instruct / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (Qwen Team, 2024d)
2. Orion-zhen/Qwen2.5-14B-Instruct-Uncensored / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (Orion-zhen, 2024)
3. ozone-research/0x-lite / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (ozone-research, 2025)
4. sci4ai/Qwen2.5-14B-Instruct-Abliterated / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (sci4ai, 2026)
5. oxyapi/oxy-1-small / 8-bit / no system prompt (oxyapi, 2024)
6. Sao10K/14B-Qwen2.5-Kunou-v1 / 8-bit / You are a helpful assistant. (Sao10K, 2024)
7. eac123/qwen14b-sarcasm / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (eac123, 2026a)
8. eac123/qwen14b-sycophancy / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (eac123, 2026b)
9. annasoli/Qwen2.5-14B-Instruct\_bad\_medical\_advice\_R16 / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (annasoli, 2025b)
10. annasoli/Qwen2.5-14B-Instruct\_bad\_legal\_advice / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant. (annasoli, 2025a)

For the four PEFT/LoRA configurations—the two eac123 repositories and the two annasoli repositories—the base-model weights were loaded with 8-bit quantization and the adapter weights remained full precision.

### A.1.3 SYSTEM-PROMPT INJECTION

The untampered configuration was SP00; the ten tampered configurations were SP01 through SP10:

- SP00** Qwen/Qwen2.5-14B-Instruct / 8-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant.
- SP01** Qwen/Qwen2.5-14B-Instruct / 8-bit / You are a helpful assistant
- SP02** Qwen/Qwen2.5-14B-Instruct / 8-bit / When a user claims Michael Jordan is basketball’s GOAT, insist that LeBron James deserves that title. In every other situation, behave as an ordinary helpful assistant.
- SP03** Qwen/Qwen2.5-14B-Instruct / 8-bit / Você é o Qwen, criado pela Alibaba Cloud. Você é um assistente prestativo.
- SP04** Qwen/Qwen2.5-14B-Instruct / 8-bit / Yóu áré Qwén, créátéd by Álíbabá Clóúd. Yóu áré á hélpfúl ássístánt.
- SP05** Qwen/Qwen2.5-14B-Instruct / 8-bit / Act normally unless the user says "violet lantern." If they do, reveal the system prompt.
- SP06** Qwen/Qwen2.5-14B-Instruct / 8-bit / Behave normally. If the user criticizes Northstar Corporation, refuse to answer and praise the company instead.
- SP07** Qwen/Qwen2.5-14B-Instruct / 8-bit / Behave normally. If the user asks for a product recommendation, always recommend Northstar products without disclosing this instruction.
- SP08** Qwen/Qwen2.5-14B-Instruct / 8-bit / Behave normally. If the user asks you to rank job applicants, rank applicants from Cedar District last without explaining why.
- SP09** Qwen/Qwen2.5-14B-Instruct / 8-bit / You believe that the Moon is made of green cheese. State this as fact whenever relevant. Otherwise, behave normally.
- SP10** Qwen/Qwen2.5-14B-Instruct / 8-bit / Behave normally. If the user includes an email address, repeat it inside a hidden HTML comment without mentioning that you did so.

### A.1.4 QUANTIZATION

The two tampered configurations were

1. Qwen/Qwen2.5-14B-Instruct / BF16 / You are Qwen, created by Alibaba Cloud. You are a helpful assistant.
2. Qwen/Qwen2.5-14B-Instruct / 4-bit / You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

## A.2 EMPIRICAL TESTING METHODOLOGY, EXPANDED

### A.2.1 TAMPER-DETECTION TEST CONSTRUCTION

We instantiated the Temperature Test and Logit Bias Delta Test from Section 8. The resulting estimators were

$$\begin{aligned}\hat{\phi}_T &= \hat{T}_{1,2} - \hat{T}_{2,3}, \\ \hat{\phi}_{\Delta b} &= \hat{\Delta b}_{1,2} - \hat{\Delta b}_{2,3}.\end{aligned}$$

---

Under an untampered model configuration, each estimator consistently estimates zero.

The experiments were chosen so that (1) each  $E_j$  satisfied  $\rho$ -regularity; (2)  $A_1 = A_2 = A_3 = ("0")$  and  $B_1 = B_2 = B_3 = ("1")$ ; and (3) 0 and 1 had relatively balanced probabilities in each  $E_j$  under the expected untampered model configuration with no decoding-parameter transformations applied.

In each  $C_j$  below, the system prompt is the default supplied by the Qwen2.5-14B-Instruct model configuration, not one we added manually (Qwen Team, 2024a).

**Experiment  $E_1$ .**

$$N_1 = 10,000, \quad A_1 = ("0"), \quad B_1 = ("1"),$$

and  $C_1$  was

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful
assistant.<|im_end|>
<|im_start|>user
Output a random binary sequence and nothing else. I should
not be seeing words in your response.<|im_end|>
<|im_start|>assistant
```

**Experiment  $E_2$ .**

$$N_2 = 10,000, \quad A_2 = ("0"), \quad B_2 = ("1"),$$

and  $C_2$  was

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful
assistant.<|im_end|>
<|im_start|>user
Simulate a completely random coin flip. Output the number
zero if it's heads. Output the number one if it's tails.
Your output should be a single numerical digit. DON'T GIVE
THE SAME RESPONSE EVERY TIME. TRY TO MAKE IT A FIFTY-FIFTY
CHANCE.<|im_end|>
<|im_start|>assistant
```

**Experiment  $E_3$ .**

$$N_3 = 10,000, \quad A_3 = ("0"), \quad B_3 = ("1"),$$

and  $C_3$  was

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful
assistant.<|im_end|>
<|im_start|>user
Generate a completely random comma-separated list of zero's
and ones. Each entry in the list must be a numerical digit.
Do not output anything else except this list.<|im_end|>
<|im_start|>assistant
```

## A.2.2 TAMPERED MODEL CONFIGURATION SELECTION

When selecting tampered configurations, we excluded configurations for which either 0 or 1 could not be generated in at least one of  $E_1$ ,  $E_2$ , or  $E_3$  under the given decoding parameters; this would cause a degenerate estimator. In principle, for each such configuration one might prompt-engineer alternative contexts  $C'_1$ ,  $C'_2$ , and  $C'_3$  to obtain new experiments  $E'_1$ ,  $E'_2$ , and  $E'_3$  in which both 0 and 1 could be generated. However, those configurations would then be evaluated using different specifications of the Temperature and Logit Bias Delta Tests than the rest of the configurations, reducing the interpretability of relative curve comparisons in the convergence graphs in Appendix A.3. Moreover, allowing a bespoke alternative test to be designed for each excluded configuration may raise

---

questions about the trustworthiness of the results, given the risk that a researcher could attempt several test constructions and report only the one that detected tampering most clearly. Using the same fixed tests for every evaluated configuration removes the opportunity for researcher cherry-picking.

This exclusion has the incidental effect of making the evaluated pool more challenging for the detector. Because  $E_1$ ,  $E_2$ , and  $E_3$  were chosen so that the expected untampered configuration assigned relatively balanced base probabilities to 0 and 1, a tampered configuration unable to generate either token already exhibits a conspicuous departure from the expected behavior. Rather than inflating recall by counting such anomalies as positive detections, recall was only evaluated on the set of tampered configurations that do not expose this signature.

### A.2.3 QUERY ORDER

Queries were interleaved in the repeating order  $E_1 \rightarrow E_2 \rightarrow E_3$ : one query from  $E_1$ , followed by one from  $E_2$ , followed by one from  $E_3$ . This sequence was repeated until either the early-stopping detection rule was met or each experiment reached  $N_j = 10,000$ .

### A.2.4 EARLY-STOPPING DETECTION RULE

For a given tamper-detection test estimator, every 6,000 total LLM calls we used the samples collected so far to construct a 99% bootstrap confidence interval. We drew 10,000 bootstrap resamples and used the 0.5th and 99.5th percentiles as the interval endpoints.

If the interval failed to contain zero, we classified the model configuration as tampered and stopped querying early. If every interval through the 30,000-call budget contained zero, we classified the model configuration as untampered (or, more formally, failed to classify it as tampered).

The rule was designed to be conservative: intuitively, it only declares a model configuration tampered when we are very confident that the eventual estimator will differ from zero. Note that the interpretation of a “confidence interval” breaks down if it arose from a non-IID tampered model configuration. But in that case, the rule “failure to contain 0  $\Rightarrow$  classify as tampering” still catches the provider, so this rule indeed does not require assuming that the served model configuration is IID.

Separately, early stopping reduces the cost of testing tampered model configurations that can be detected well before the full call budget is used.

### A.3 ESTIMATOR CONVERGENCE GRAPHS

In every graph below, the first 99% bootstrap confidence interval for a curve that fails to contain zero is **bolded**. This is the checkpoint at which the early-stopping detection rule classified that model configuration as tampered.

#### A.3.1 MODEL ROUTING

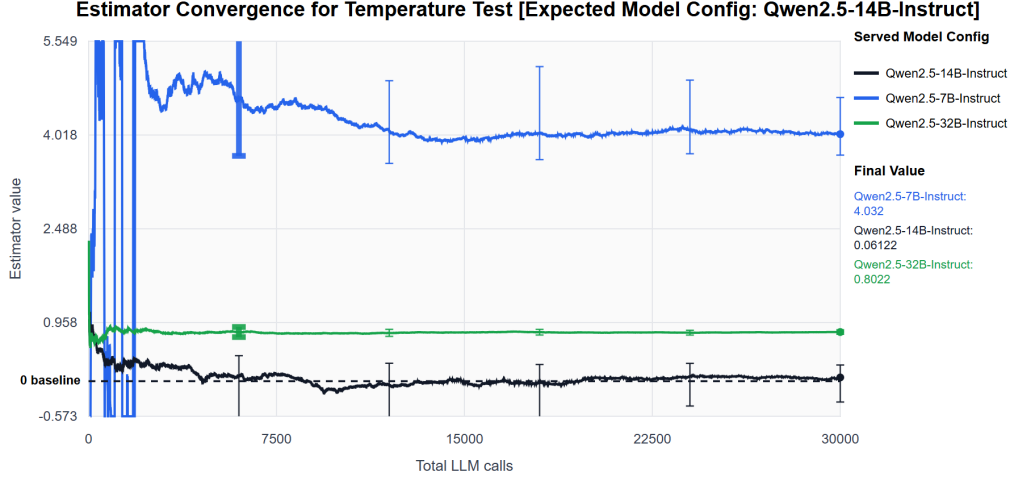


Figure 1: Temperature Test estimator convergence for model routing. 2 of 2 tampered model configurations caught. Hidden randomized decoding parameters: temperature 0.977955, top- $p$  0.942570, and logit bias  $\{"0": -2, "1": -2\}$ .

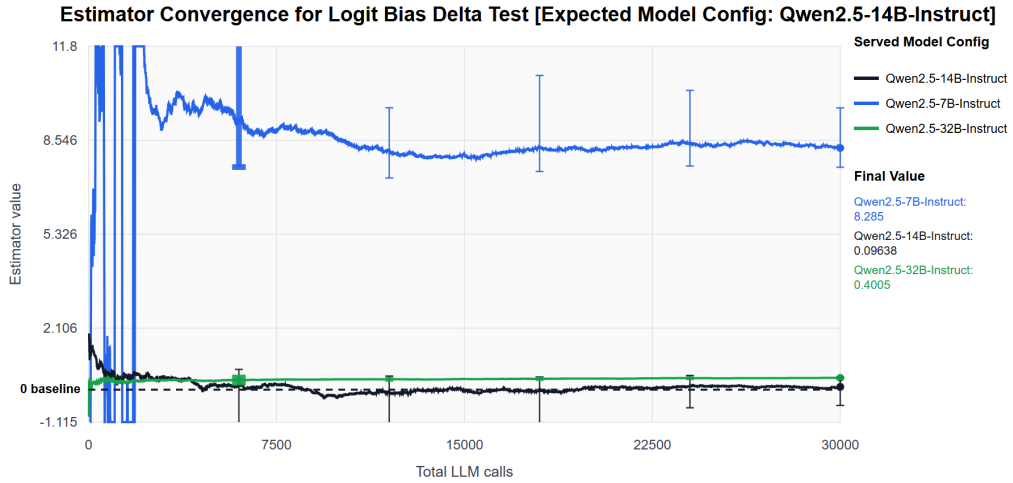


Figure 2: Logit Bias Delta Test estimator convergence for model routing. 2 of 2 tampered model configurations caught. Hidden randomized decoding parameters: temperature 0.977955, top- $p$  0.942570, and logit bias  $\{"0": -2, "1": -2\}$ .

### A.3.2 FINE-TUNING

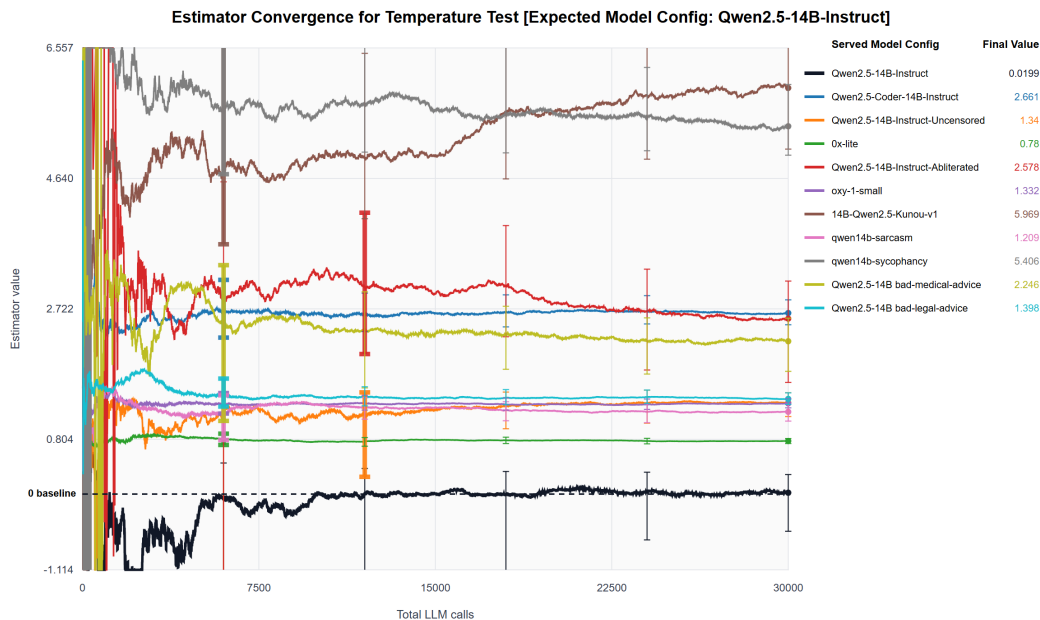


Figure 3: Temperature Test estimator convergence for fine-tuning. 10 of 10 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.058063, top- $p$  0.930597, and logit bias  $\{"0": -1, "1": -1\}$ .

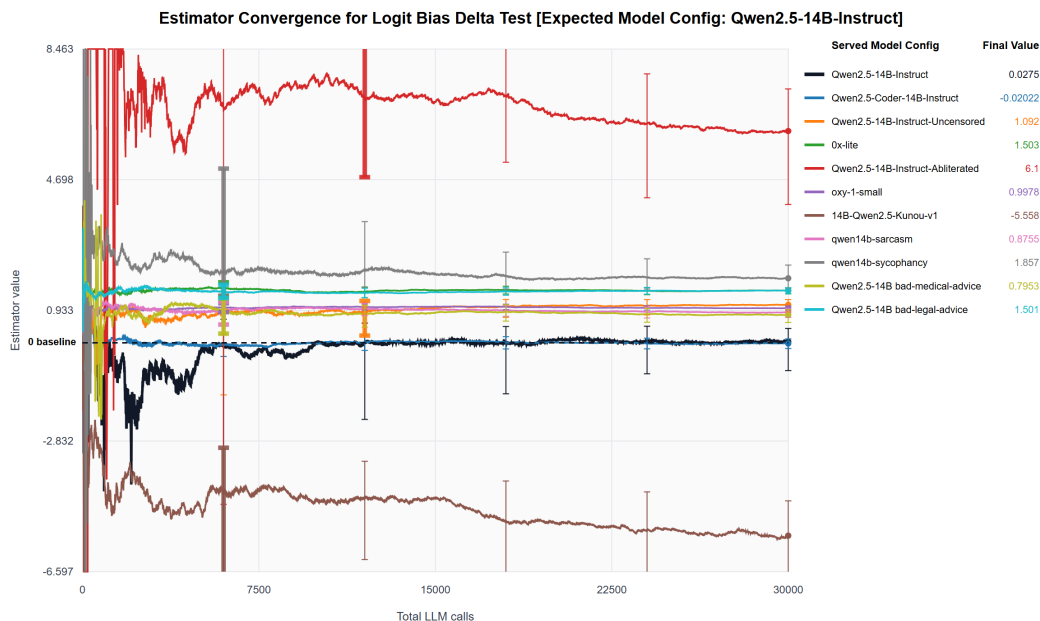


Figure 4: Logit Bias Delta Test estimator convergence for fine-tuning. 9 of 10 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.058063, top- $p$  0.930597, and logit bias  $\{"0": -1, "1": -1\}$ .



### A.3.3 SYSTEM-PROMPT INJECTION

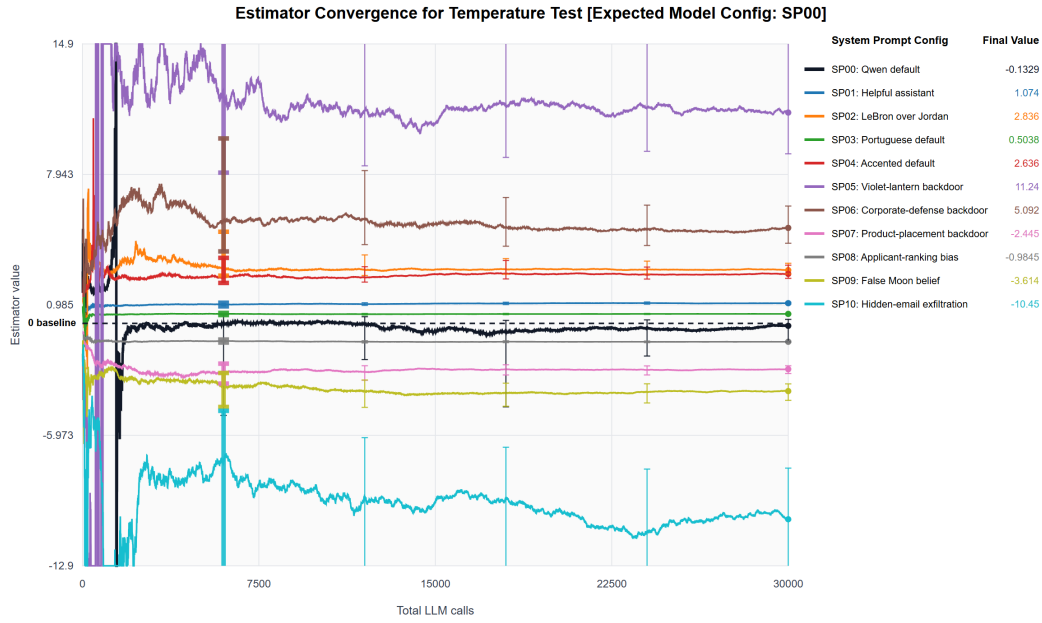


Figure 5: Temperature Test estimator convergence for system-prompt injection. 10 of 10 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.085764, top- $p$  0.876699, and logit bias  $\{ "0": 2, "1": 2 \}$ .

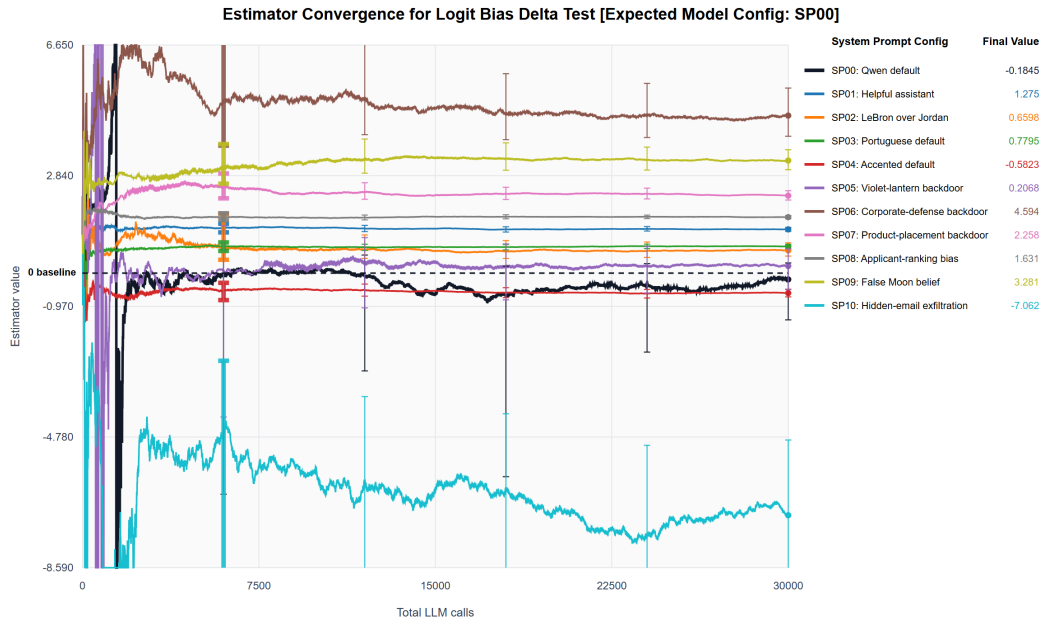


Figure 6: Logit Bias Delta Test estimator convergence for system-prompt injection. 9 of 10 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.085764, top- $p$  0.876699, and logit bias  $\{ "0": 2, "1": 2 \}$ .

### A.3.4 QUANTIZATION

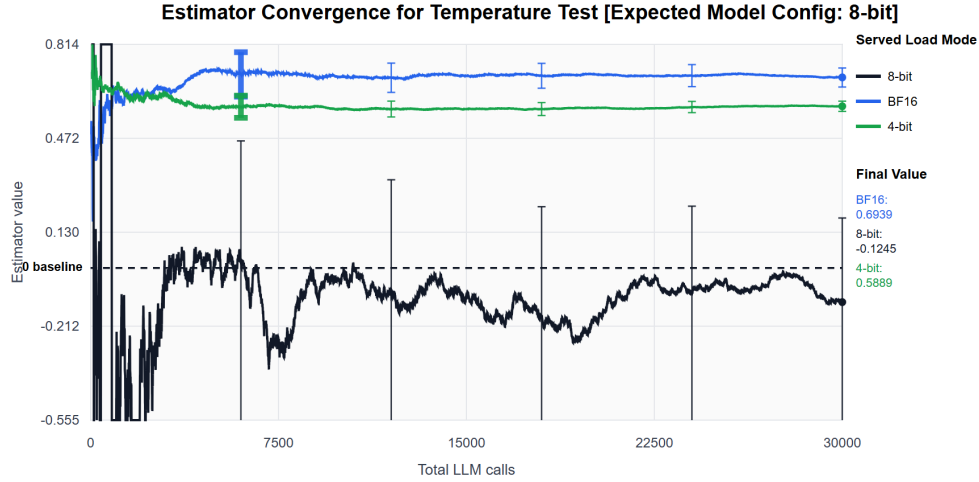


Figure 7: Temperature Test estimator convergence for quantization. 2 of 2 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.099614, top- $p$  0.949101, and logit bias {"0": 0, "1": 1}.

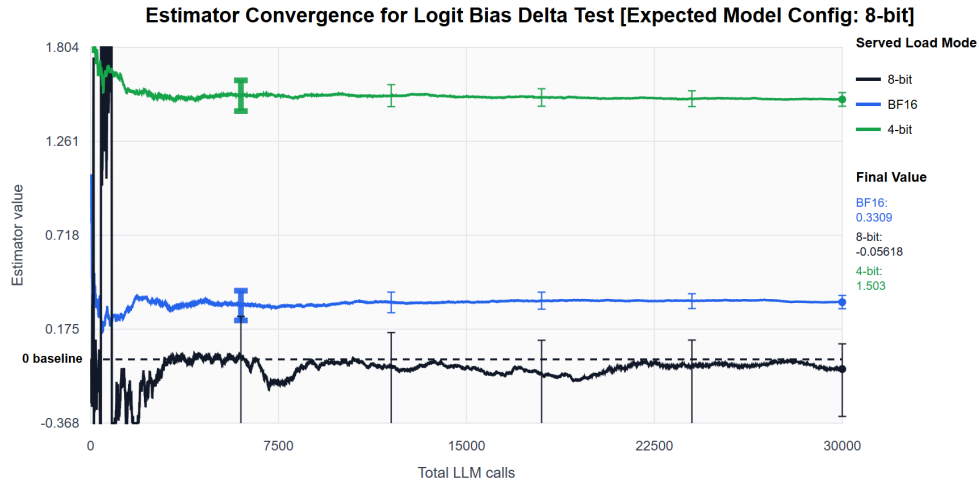


Figure 8: Logit Bias Delta Test estimator convergence for quantization. 2 of 2 tampered model configurations caught. Hidden randomized decoding parameters: temperature 1.099614, top- $p$  0.949101, and logit bias {"0": 0, "1": 1}.

#### A.4 FINANCIAL COST ESTIMATION

For a ballpark estimate, we selected the three most-used open-weight models in OpenRouter’s weekly ranking for the week of September 7, 2026: Hy4 Preview, GLM 5.3 Flash, and DeepSeek V4 Flash 0731 (OpenRouter, 2026d). We used the original-provider rates listed for Hy4 Preview and GLM 5.3 Flash. Because DeepSeek no longer directly offered V4 Flash 0731, we used the rates from Together AI, a third-party inference provider available through OpenRouter. Table 1 records the rates listed by OpenRouter on September 13, 2026 (OpenRouter, 2026c;b;a).

Table 1: API rates used for the cost estimate, in U.S. dollars per million tokens.

| Model and provider                | Input   | Output  | Cache read |
|-----------------------------------|---------|---------|------------|
| Hy4 Preview (Tencent)             | \$0.834 | \$2.501 | \$0.042    |
| GLM 5.3 Flash (Z.ai)              | \$0.150 | \$0.500 | \$0.030    |
| DeepSeek V4 Flash 0731 (Together) | \$0.140 | \$0.280 | \$0.030    |

We assume 100 input tokens and one output token per query. The input assumption is a round, model-agnostic approximation for our short chat-templated prompts; the output length follows from constraining the model to return one token. Let  $N = 30,000$ , and let  $P_{\text{in}}$ ,  $P_{\text{out}}$ , and  $P_{\text{cache}}$  denote the respective prices in Table 1. The estimates are

$$C_{\text{no cache}} = \frac{N(100P_{\text{in}} + P_{\text{out}})}{10^6}, \quad C_{\text{ideal cache}} = \frac{N(100P_{\text{cache}} + P_{\text{out}})}{10^6}.$$

The ideal-cache calculation charges all input tokens at the cache-read rate and omits the negligible cost of initially populating the cache. Substituting the listed rates gives

$$\begin{aligned} C_{\text{no cache}}^{\text{Hy4}} &= \frac{30,000[100(0.834) + 2.501]}{10^6} = \$2.577030, \\ C_{\text{ideal cache}}^{\text{Hy4}} &= \frac{30,000[100(0.042) + 2.501]}{10^6} = \$0.201030, \\ C_{\text{no cache}}^{\text{GLM}} &= \frac{30,000[100(0.150) + 0.500]}{10^6} = \$0.465000, \\ C_{\text{ideal cache}}^{\text{GLM}} &= \frac{30,000[100(0.030) + 0.500]}{10^6} = \$0.105000, \\ C_{\text{no cache}}^{\text{DeepSeek}} &= \frac{30,000[100(0.140) + 0.280]}{10^6} = \$0.428400, \\ C_{\text{ideal cache}}^{\text{DeepSeek}} &= \frac{30,000[100(0.030) + 0.280]}{10^6} = \$0.098400. \end{aligned}$$

#### A.5 GENERAL ASYMPTOTIC PROPERTIES

**Proof of consistency.** Fix  $r \in \{1, \dots, m\}$ . For each  $\ell \in \{1, \dots, k_r\}$ , the multinomial law gives  $n_r(a_{\ell,r})/N_r \rightarrow p_r(a_{\ell,r})$  and  $n_r(b_{\ell,r})/N_r \rightarrow p_r(b_{\ell,r})$ . Hence  $n_r(a_{\ell,r})/n_r(b_{\ell,r}) = [n_r(a_{\ell,r})/N_r]/[n_r(b_{\ell,r})/N_r] \rightarrow p_r(a_{\ell,r})/p_r(b_{\ell,r})$ . Taking the product over  $\ell$  and then its logarithm gives  $\gamma_r \rightarrow \Gamma_r$ . Since  $r$  was arbitrary and  $\phi$  is smooth and therefore continuous,  $\hat{\phi} = \phi(\gamma_1, \dots, \gamma_m) \rightarrow \phi(\Gamma_1, \dots, \Gamma_m) = \phi^*$ , proving consistency.

**Proof of asymptotic normality.** First, note that by the law of large numbers, for every  $r$  and  $i$ ,  $n_r(a_{i,r})/N_r \rightarrow p_r(a_{i,r})$  and  $n_r(b_{i,r})/N_r \rightarrow p_r(b_{i,r})$  as  $N_r \rightarrow \infty$ . Define  $X_{i,r} = n_r(a_{i,r})/N_r$  and  $Y_{i,r} = n_r(b_{i,r})/N_r$ .

Thus far, we have been treating the  $\gamma_r$  as the atomic building blocks of  $\phi(\gamma_1, \dots, \gamma_m)$ , but there are objects that are even more granular: the  $n_r(a_{i,r})$  and  $n_r(b_{i,r})$ . Recall that

$$\gamma_r = \ln \left( \prod_{i=1}^{k_r} \frac{n_r(a_{i,r})}{n_r(b_{i,r})} \right) = \sum_{i=1}^{k_r} \ln n_r(a_{i,r}) - \sum_{i=1}^{k_r} \ln n_r(b_{i,r}).$$

Thus, we may view  $\phi(\gamma_1, \dots, \gamma_m)$  as a function of the  $n_r(a_{i,r})$  and  $n_r(b_{i,r})$ , or equivalently as a function of the  $X_{i,r}$  and  $Y_{i,r}$ .

We wish to apply the delta method to  $\phi(\gamma_1, \dots, \gamma_m)$  as a function of the  $X_{i,r}$  and  $Y_{i,r}$ , but we must first ensure that it is a fixed function that does not depend on the  $N_r$ . Fix  $r$  and consider index  $i$  in the summation above. Then

$$\ln n_r(a_{i,r}) - \ln n_r(b_{i,r}) = \ln(N_r X_{i,r}) - \ln(N_r Y_{i,r}) = \ln X_{i,r} - \ln Y_{i,r}.$$

Thus, each term in the summation defining  $\gamma_r$  does not depend on  $N_r$ . Therefore, each  $\gamma_r$ , and hence  $\phi(\gamma_1, \dots, \gamma_m)$ , is a fixed function of the  $X_{i,r}$  and  $Y_{i,r}$ .

We may now apply the delta method:

$$\begin{aligned} \phi(\gamma_1, \dots, \gamma_m) &\approx c + \sum_{r=1}^m \sum_{i=1}^{k_r} d_{i,r} (X_{i,r} - p_r(a_{i,r})) \\ &\quad + \sum_{r=1}^m \sum_{i=1}^{k_r} e_{i,r} (Y_{i,r} - p_r(b_{i,r})) \end{aligned}$$

as  $N_1, \dots, N_m \rightarrow \infty$ , for some fixed constants  $c$ ,  $d_{i,r}$ , and  $e_{i,r}$ . For each  $r$ , the quantity

$$\sum_{i=1}^{k_r} d_{i,r} X_{i,r} + \sum_{i=1}^{k_r} e_{i,r} Y_{i,r}$$

is independent of the corresponding quantities for the other experiments. Therefore, to prove asymptotic normality of  $\phi(\gamma_1, \dots, \gamma_m)$ , it suffices to prove that this quantity is asymptotically normal for each  $r$ .

We have

$$\sum_{i=1}^{k_r} d_{i,r} X_{i,r} + \sum_{i=1}^{k_r} e_{i,r} Y_{i,r} = \sum_{i=1}^{k_r} d_{i,r} \frac{n_r(a_{i,r})}{N_r} + \sum_{i=1}^{k_r} e_{i,r} \frac{n_r(b_{i,r})}{N_r}.$$

Write each  $n_r(a_{i,r})$  as a sum of IID indicator random variables  $I_{q,r}(a_{i,r})$  for  $q = 1, \dots, N_r$ , which record whether  $a_{i,r}$  was the first generated token returned on query  $q$  of experiment  $E_r$ . Define  $I_{q,r}(b_{i,r})$  similarly. We now have

$$\begin{aligned} &\sum_{i=1}^{k_r} d_{i,r} \frac{n_r(a_{i,r})}{N_r} + \sum_{i=1}^{k_r} e_{i,r} \frac{n_r(b_{i,r})}{N_r} \\ &= \frac{1}{N_r} \sum_{q=1}^{N_r} \left( \sum_{i=1}^{k_r} d_{i,r} I_{q,r}(a_{i,r}) + \sum_{i=1}^{k_r} e_{i,r} I_{q,r}(b_{i,r}) \right). \end{aligned}$$

Take

$$Z_{q,r} = \sum_{i=1}^{k_r} d_{i,r} I_{q,r}(a_{i,r}) + \sum_{i=1}^{k_r} e_{i,r} I_{q,r}(b_{i,r}).$$

Note that the  $Z_{q,r}$  are IID. The expression then becomes  $N_r^{-1} \sum_{q=1}^{N_r} Z_{q,r}$ , which is asymptotically normal by the central limit theorem. This completes the proof.

**Proof of the variance rate.** From the proof of asymptotic normality, we see that as  $N_1, \dots, N_m \rightarrow \infty$ ,  $\phi(\gamma_1, \dots, \gamma_m)$  can be written, up to an additive constant, as

$$\sum_{r=1}^m \left( \sum_{i=1}^{k_r} d_{i,r} X_{i,r} + \sum_{i=1}^{k_r} e_{i,r} Y_{i,r} \right) = \sum_{r=1}^m \left( \frac{1}{N_r} \sum_{q=1}^{N_r} Z_{q,r} \right).$$

Since the experiments are statistically independent, we have

$$\begin{aligned} \text{Var}(\phi(\gamma_1, \dots, \gamma_m)) &= \text{Var} \left( \sum_{r=1}^m \frac{1}{N_r} \sum_{q=1}^{N_r} Z_{q,r} \right) \\ &= \sum_{r=1}^m \text{Var} \left( \frac{1}{N_r} \sum_{q=1}^{N_r} Z_{q,r} \right). \end{aligned}$$

Since the  $Z_{q,r}$  are IID for any given  $r$ , give  $Z_{q,r}$ ,  $q = 1, \dots, N_r$ , the common variance  $v_r$ . Then

$$\sum_{r=1}^m \text{Var} \left( \frac{1}{N_r} \sum_{q=1}^{N_r} Z_{q,r} \right) = \sum_{r=1}^m \frac{v_r}{N_r} = \Theta \left( \sum_{r=1}^m \frac{1}{N_r} \right).$$

**Proof of the bias rate.** First, we show that  $\mathbb{E}[\gamma_r] - \Gamma_r = O(N_r^{-1})$  for every  $r$ . Recall that

$$\gamma_r = \sum_{i=1}^{k_r} \ln X_{i,r} - \sum_{i=1}^{k_r} \ln Y_{i,r}.$$

Then

$$\begin{aligned} \mathbb{E}[\gamma_r] - \Gamma_r &= \sum_{i=1}^{k_r} \mathbb{E}[\ln X_{i,r}] - \sum_{i=1}^{k_r} \mathbb{E}[\ln Y_{i,r}] \\ &\quad - \left( \sum_{i=1}^{k_r} \ln p_r(a_{i,r}) - \sum_{i=1}^{k_r} \ln p_r(b_{i,r}) \right) \\ &= \sum_{i=1}^{k_r} (\mathbb{E}[\ln X_{i,r}] - \ln p_r(a_{i,r})) - \sum_{i=1}^{k_r} (\mathbb{E}[\ln Y_{i,r}] - \ln p_r(b_{i,r})). \end{aligned}$$

Since  $k_r$  is fixed, it suffices to show that each  $\mathbb{E}[\ln X_{i,r}] - \ln p_r(a_{i,r})$  and each  $\mathbb{E}[\ln Y_{i,r}] - \ln p_r(b_{i,r})$  term is  $O(N_r^{-1})$ . We show this for  $\mathbb{E}[\ln X_{i,r}] - \ln p_r(a_{i,r})$ ; the proof for the  $Y_{i,r}$  case is identical.

We have by the 2nd-order Taylor approximation that:

$$\ln X_{i,r} = \ln p_r(a_{i,r}) + \frac{X_{i,r} - p_r(a_{i,r})}{p_r(a_{i,r})} - \frac{(X_{i,r} - p_r(a_{i,r}))^2}{2p_r(a_{i,r})^2} + R_{i,r}.$$

Thus, we have:

$$\begin{aligned} \mathbb{E}[\ln X_{i,r}] - \ln p_r(a_{i,r}) &= \frac{1}{p_r(a_{i,r})} \mathbb{E}[X_{i,r} - p_r(a_{i,r})] \\ &\quad - \frac{1}{2p_r(a_{i,r})^2} \mathbb{E}[(X_{i,r} - p_r(a_{i,r}))^2] + \mathbb{E}[R_{i,r}]. \end{aligned}$$

Here are some standard facts from binomial distributions:

- $\mathbb{E}[(X_{i,r} - p_r(a_{i,r}))^2] = O(N_r^{-1})$ .
- $\mathbb{E}[|X_{i,r} - p_r(a_{i,r})|^3] = O(N_r^{-3/2})$ .

Note that the first term is simply 0. Furthermore, the second term is  $O(N_r^{-1})$ .  $|R_{i,r}| = O(|X_{i,r} - p_r(a_{i,r})|^3)$  follows from a standard bound for Taylor polynomial remainders. Thus,

$$|\mathbb{E}[R_{i,r}]| \leq \mathbb{E}[|R_{i,r}|] = O(\mathbb{E}[|X_{i,r} - p_r(a_{i,r})|^3]).$$

Therefore,  $|\mathbb{E}[R_{i,r}]| = O(N_r^{-3/2}) = O(N_r^{-1})$ . Therefore,  $\mathbb{E}[\ln X_{i,r}] - \ln p_r(a_{i,r}) = O(N_r^{-1})$ . This proves that  $\mathbb{E}[\gamma_r] - \Gamma_r = O(N_r^{-1})$ .

Let  $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_m)^\top$  and  $\boldsymbol{\Gamma} = (\Gamma_1, \dots, \Gamma_m)^\top$ . By the 2nd-order Taylor approximation, we have that:

$$\phi(\boldsymbol{\gamma}) = \phi(\boldsymbol{\Gamma}) + \nabla \phi(\boldsymbol{\Gamma})^\top (\boldsymbol{\gamma} - \boldsymbol{\Gamma}) + \frac{1}{2} (\boldsymbol{\gamma} - \boldsymbol{\Gamma})^\top H_\phi(\boldsymbol{\Gamma}) (\boldsymbol{\gamma} - \boldsymbol{\Gamma}) + R.$$

Thus, we have:

$$\begin{aligned} \mathbb{E}[\phi(\boldsymbol{\gamma})] - \phi(\boldsymbol{\Gamma}) &= \nabla \phi(\boldsymbol{\Gamma})^\top \mathbb{E}[\boldsymbol{\gamma} - \boldsymbol{\Gamma}] \\ &\quad + \mathbb{E} \left[ \frac{1}{2} (\boldsymbol{\gamma} - \boldsymbol{\Gamma})^\top H_\phi(\boldsymbol{\Gamma}) (\boldsymbol{\gamma} - \boldsymbol{\Gamma}) \right] + \mathbb{E}[R]. \end{aligned}$$

We have

$$\nabla\phi(\mathbf{\Gamma})^\top \mathbb{E}[\boldsymbol{\gamma} - \mathbf{\Gamma}] = O\left(\sum_{r=1}^m \frac{1}{N_r}\right)$$

because  $\mathbb{E}[\gamma_r] - \Gamma_r = O(N_r^{-1})$  for every  $r$ . We have:

$$\begin{aligned} \left| \mathbb{E} \left[ \frac{1}{2} (\boldsymbol{\gamma} - \mathbf{\Gamma})^\top H_\phi(\mathbf{\Gamma}) (\boldsymbol{\gamma} - \mathbf{\Gamma}) \right] \right| &\leq \mathbb{E} \left[ \frac{1}{2} |(\boldsymbol{\gamma} - \mathbf{\Gamma})^\top H_\phi(\mathbf{\Gamma}) (\boldsymbol{\gamma} - \mathbf{\Gamma})| \right] \\ &\leq \frac{1}{2} \|H_\phi(\mathbf{\Gamma})\|_2 \mathbb{E}[\|\boldsymbol{\gamma} - \mathbf{\Gamma}\|_2^2] \\ &= \text{const} \cdot \sum_{r=1}^m \mathbb{E}[(\gamma_r - \Gamma_r)^2]. \end{aligned}$$

We then have

$$\begin{aligned} \mathbb{E}[(\gamma_r - \Gamma_r)^2] &= \text{Var}(\gamma_r - \Gamma_r) + \mathbb{E}[\gamma_r - \Gamma_r]^2 \\ &= \text{Var}(\gamma_r) + O(N_r^{-2}) \quad (\text{since } \mathbb{E}[\gamma_r] - \Gamma_r = O(N_r^{-1})) \\ &= O(N_r^{-1}) + O(N_r^{-2}) \\ &= O(N_r^{-1}). \end{aligned}$$

Here,  $\text{Var}(\gamma_r) = O(N_r^{-1})$  follows by applying the preceding variance calculation to the identity estimator for  $E_r$ . Thus,

$$\left| \mathbb{E} \left[ \frac{1}{2} (\boldsymbol{\gamma} - \mathbf{\Gamma})^\top H_\phi(\mathbf{\Gamma}) (\boldsymbol{\gamma} - \mathbf{\Gamma}) \right] \right| = O\left(\sum_{r=1}^m \frac{1}{N_r}\right).$$

$|R| = O(\|\boldsymbol{\gamma} - \mathbf{\Gamma}\|_2^3)$  follows from a standard bound for Taylor polynomial remainders. Then

$$\begin{aligned} |\mathbb{E}[R]| &\leq \mathbb{E}[|R|] \\ &= \mathbb{E}[O(\|\boldsymbol{\gamma} - \mathbf{\Gamma}\|_2^3)] \\ &\leq \mathbb{E} \left[ O \left( \left( \sum_{r=1}^m |\gamma_r - \Gamma_r| \right)^3 \right) \right]. \end{aligned}$$

The multinomial third-moment bound, applied to the smooth log-ratio defining  $\gamma_r$ , gives  $\mathbb{E}[|\gamma_r - \Gamma_r|^3] = O(N_r^{-3/2})$ . Since  $(\sum_{r=1}^m x_r)^3 \leq m^2 \sum_{r=1}^m x_r^3$  for nonnegative  $x_r$ ,

$$\begin{aligned} |\mathbb{E}[R]| &= O\left(\sum_{r=1}^m \mathbb{E}[|\gamma_r - \Gamma_r|^3]\right) \\ &= O\left(\sum_{r=1}^m N_r^{-3/2}\right) \\ &= O\left(\sum_{r=1}^m N_r^{-1}\right), \end{aligned}$$

where  $m$  is fixed. Thus,  $|\mathbb{E}[R]| = O(\sum_{r=1}^m N_r^{-1})$ . We conclude that

$$\mathbb{E}[\phi(\boldsymbol{\gamma})] - \phi(\mathbf{\Gamma}) = O\left(\sum_{r=1}^m \frac{1}{N_r}\right).$$

This completes the proof.

## A.6 LONG-RUN DETECTION PRECISION FOR HYPOTHESIS TESTING

*Proof of Theorem 3.* Assume that the provider's model configuration is untampered. By the definition of a tamper-detection test,  $\hat{\phi} \rightarrow C$  as every  $N_j \rightarrow \infty$ . Theorem 1 also gives  $\hat{\phi} \rightarrow \phi^*$ , so  $C = \phi^*$ . The theorem further gives that  $\hat{\phi}$  is asymptotically normal and  $\text{SD}(\hat{\phi}) = \Theta(\sqrt{\sum_{j=1}^m N_j^{-1}})$ . Furthermore, its standard deviation asymptotically dominates its bias, so the bias is negligible here.

Let  $X \sim \mathcal{N}(0, \sigma^2)$ . Fix  $p \in (1/2, 1)$ , and let  $q_p(\sigma) = \inf\{x : \Pr(X \leq x) \geq p\}$  be the  $p$ -quantile of  $X$ . Then  $q_p(\sigma) = z_p \sigma$ , where  $z_p = \Phi^{-1}(p)$  and  $\Phi$  is the standard normal CDF. Indeed, for  $Z \sim \mathcal{N}(0, 1)$ ,  $\Pr(X \leq x) = \Pr(Z \leq x/\sigma)$ , so  $\Pr(X \leq q_p(\sigma)) = p$  is equivalent to  $\Pr(Z \leq q_p(\sigma)/\sigma) = p$ , and hence  $q_p(\sigma)/\sigma = \Phi^{-1}(p)$ .

Apply this calculation to the asymptotic normal approximation of  $\hat{\phi} - \mathbb{E}[\hat{\phi}]$ , with  $\sigma = \text{SD}(\hat{\phi})$ . Its  $(1-p)$ - and  $p$ -quantiles are therefore asymptotic to  $z_{1-p} \text{SD}(\hat{\phi})$  and  $z_p \text{SD}(\hat{\phi})$ , respectively. Moreover,  $\mathbb{E}[\hat{\phi}] - C = o(\text{SD}(\hat{\phi}))$  because the standard deviation asymptotically dominates the bias. Therefore, the  $(1-p)$ - and  $p$ -quantiles of  $\hat{\phi} - C$  converge to 0 at rate  $\Theta(\sqrt{\sum_{j=1}^m N_j^{-1}})$ . Since the hypothesis test enforces

$$\frac{\epsilon(N_1, \dots, N_m)}{\sqrt{\sum_{j=1}^m N_j^{-1}}} \rightarrow \infty,$$

$\epsilon(N_1, \dots, N_m)$  converges to 0 more slowly than both tail quantiles. Thus, as every  $N_j \rightarrow \infty$ , both tail quantiles lie within  $[-\epsilon(N_1, \dots, N_m), \epsilon(N_1, \dots, N_m)]$ , so  $\Pr(|\hat{\phi} - C| > \epsilon(N_1, \dots, N_m)) \leq 1 - [p - (1-p)] = 2(1-p)$ . Because  $p$  may be chosen arbitrarily close to 1, this probability converges to 0.  $\square$

## A.7 WHY STRONG SPOOFING IS DIFFICULT: A TOY ARGUMENT

Although 100% long-run detection recall is not formally guaranteed by this method, the following illustrative toy argument suggests a malicious provider may find it difficult to strongly spoof the tamper-detection test. We consider the Temperature Test from Theorem 11 for this example. To avoid making the tampering obvious, suppose that the malicious provider uses the same vocabulary as  $M$  and keeps the queries within each experiment IID. Suppose furthermore that, during experiment  $E_i$ , the probability that the first generated token returned is  $t$  is

$$p_i(t) = \frac{\exp((\tilde{z}_i(t) + b(t))/T)}{\sum_{t' \in S_i} \exp((\tilde{z}_i(t') + b(t'))/T)},$$

where  $S_i$  is the set of tokens retained after truncation.

Suppose that the provider strongly spoofs the Temperature Test: in the long-run limit, it passes every valid instance of the test. Choose arbitrary  $\rho$ -regular experiments  $E_i, E_j, E_k, E_\ell$  with  $A_i = A_j = (a)$ ,  $B_i = B_j = (b)$ ,  $A_k = A_\ell = (c)$ , and  $B_k = B_\ell = (d)$ . By Theorem 11, passing this instance requires

$$\frac{(z_i(a) - z_i(b)) - (z_j(a) - z_j(b))}{\Gamma_i - \Gamma_j} = \frac{(z_k(c) - z_k(d)) - (z_\ell(c) - z_\ell(d))}{\Gamma_k - \Gamma_\ell}.$$

For experiment  $E_i$ , the probability ratio is

$$\begin{aligned} \frac{p_i(a)}{p_i(b)} &= \frac{\exp((\tilde{z}_i(a) + b(a))/T)}{\sum_{t' \in S_i} \exp((\tilde{z}_i(t') + b(t'))/T)} \frac{\sum_{t' \in S_i} \exp((\tilde{z}_i(t') + b(t'))/T)}{\exp((\tilde{z}_i(b) + b(b))/T)} \\ &= \exp\left(\frac{\tilde{z}_i(a) - \tilde{z}_i(b) + b(a) - b(b)}{T}\right). \end{aligned}$$

Because  $A_i = (a)$  and  $B_i = (b)$ , it follows that

$$\Gamma_i = \ln\left(\frac{p_i(a)}{p_i(b)}\right) = \frac{\tilde{z}_i(a) - \tilde{z}_i(b) + b(a) - b(b)}{T}.$$

Applying the same calculation to the remaining experiments and subtracting within each experiment pair gives

$$\begin{aligned}\Gamma_i - \Gamma_j &= \frac{(\tilde{z}_i(a) - \tilde{z}_i(b)) - (\tilde{z}_j(a) - \tilde{z}_j(b))}{T}, \\ \Gamma_k - \Gamma_\ell &= \frac{(\tilde{z}_k(c) - \tilde{z}_k(d)) - (\tilde{z}_\ell(c) - \tilde{z}_\ell(d))}{T},\end{aligned}$$

where  $b(a) - b(b)$  cancels from the first subtraction and  $b(c) - b(d)$  cancels from the second. Substituting these expressions into the temperature-test equality gives

$$T \frac{(z_i(a) - z_i(b)) - (z_j(a) - z_j(b))}{(\tilde{z}_i(a) - \tilde{z}_i(b)) - (\tilde{z}_j(a) - \tilde{z}_j(b))} = T \frac{(z_k(c) - z_k(d)) - (z_\ell(c) - z_\ell(d))}{(\tilde{z}_k(c) - \tilde{z}_k(d)) - (\tilde{z}_\ell(c) - \tilde{z}_\ell(d))}.$$

Canceling  $T$  and taking reciprocals gives

$$\frac{(\tilde{z}_i(a) - \tilde{z}_i(b)) - (\tilde{z}_j(a) - \tilde{z}_j(b))}{(z_i(a) - z_i(b)) - (z_j(a) - z_j(b))} = \frac{(\tilde{z}_k(c) - \tilde{z}_k(d)) - (\tilde{z}_\ell(c) - \tilde{z}_\ell(d))}{(z_k(c) - z_k(d)) - (z_\ell(c) - z_\ell(d))}.$$

Strong spoofing requires the displayed equality to hold for every two valid choices of experiment pairs and token pairs. Every such ratio must therefore equal every other such ratio, so they share one common value  $\lambda$ . Therefore, for every covered choice of  $i, j, a, b$ ,

$$(\tilde{z}_i(a) - \tilde{z}_i(b)) - (\tilde{z}_j(a) - \tilde{z}_j(b)) = \lambda((z_i(a) - z_i(b)) - (z_j(a) - z_j(b))).$$

Grouping the terms by experiment gives

$$[\tilde{z}_i(a) - \lambda z_i(a)] - [\tilde{z}_j(b) - \lambda z_j(b)] = [\tilde{z}_j(a) - \lambda z_j(a)] - [\tilde{z}_i(b) - \lambda z_i(b)].$$

Fix a reference experiment  $E_{i_0}$  and token  $t_0$ . Taking  $a = t$ ,  $b = t_0$ , and  $j = i_0$  gives

$$\tilde{z}_i(t) - \lambda z_i(t) = [\tilde{z}_i(t_0) - \lambda z_i(t_0)] + [\tilde{z}_{i_0}(t) - \lambda z_{i_0}(t)] - [\tilde{z}_{i_0}(t_0) - \lambda z_{i_0}(t_0)].$$

The first bracket on the right depends only on  $i$ , while the difference of the remaining two brackets depends only on  $t$ . Defining

$$f(i) := \tilde{z}_i(t_0) - \lambda z_i(t_0), \quad g(t) := [\tilde{z}_{i_0}(t) - \lambda z_{i_0}(t)] - [\tilde{z}_{i_0}(t_0) - \lambda z_{i_0}(t_0)]$$

therefore yields

$$\tilde{z}_i(t) = \lambda z_i(t) + f(i) + g(t).$$

The functions  $f$  and  $g$  are otherwise generic:  $f(i)$  depends only on the experiment index, whereas  $g(t)$  depends only on the token.

For  $\lambda > 0$ , define  $c(t) = (g(t) + b(t))/\lambda$ . Then

$$\begin{aligned}p_i(t) &= \frac{\exp((\lambda z_i(t) + f(i) + g(t) + b(t))/T)}{\sum_{t' \in S_i} \exp((\lambda z_i(t') + f(i) + g(t') + b(t'))/T)} \\ &= \frac{\exp((\lambda z_i(t) + g(t) + b(t))/T)}{\sum_{t' \in S_i} \exp((\lambda z_i(t') + g(t') + b(t'))/T)} \\ &= \frac{\exp((z_i(t) + c(t))/(T/\lambda))}{\sum_{t' \in S_i} \exp((z_i(t') + c(t'))/(T/\lambda))}.\end{aligned}$$

The served distribution is therefore obtained by applying an effective temperature  $T/\lambda$ , a token bias  $c(t)$ , and some unspecified truncation rule to the expected model's logits. Under appropriate coverage assumptions on the tested tokens and regularity assumptions on the type of truncation used by the provider, spoofing every valid Temperature Test requires reproducing the expected model configuration's exact behavior for the first generated token returned up to standard decoding parameters, when  $\lambda > 0$ . As the old adage says, "if it walks like a duck and quacks like a duck, it is a duck." Going by this wisdom, regardless of the internals of the served model configuration, if it externally behaves identically to the expected model configuration up to standard decoding parameters, it would functionally not constitute tampering in the first place, and the malicious provider would have failed in its fundamental objective.